

Gestor de Credenciales Seguro

Prácticas 5 y 6. S-TDD y Diseño Seguro

Grupo: Lara Moriel, Amelia Martínez, Enrique Pérez, Nikhil Baxani, Gael Espín, Sofía Casermeiro.

1. Introducción

En esta práctica hemos desarrollado un gestor de credenciales seguro en dos partes. En la Práctica 5 implementamos el sistema usando S-TDD, siguiendo el ciclo test → implementación → refactorización con roles rotativos. En la Práctica 6 extendimos el sistema aplicando principios SOLID y los patrones Strategy, Secure Strategy Factory y Secure Proxy for API.

El resultado final tiene 56 tests, todos en verde.

2. Registro de trabajo

2.1. Metodología: ciclos S-TDD

El trabajo de la Práctica 5 se organizó en seis ciclos, para que cada miembro del grupo participase en todas las fases del proceso de desarrollo. En cada ciclo una persona escribe los tests (en rojo), la siguiente los implementa (verde) y la siguiente refactoriza. Los roles rotan en cada iteración:

Ciclo	Hace el Test	Lo implementa	Refactorizar
1	Lara	Amelia	Enrique
2	Amelia	Enrique	Nikhil
3	Enrique	Nikhil	Gael
4	Nikhil	Gael	Sofía
5	Gael	Sofía	Lara
6	Sofía	Lara	Amelia

2.2. Práctica 5 — Ciclos S-TDD

Ciclo 1 — `anyadir_credencial`, `eliminar_credencial`

Lara escribió dos tests: uno que verifica que añadir una credencial permite recuperarla, y otro que verifica que eliminarla hace que recuperarla lance `KeyError`. Los entregó en rojo.

Amelia implementó el almacenamiento en memoria y las funciones del gestor `anyadir()`, `recuperar()`, `eliminar()`.

Enrique refactorizó el código con SonarQube, extraxo constantes y unificó el manejo de excepciones.

Ciclo 2 — acceso_con_clave_maestra_erronea, timeout_sesion_clave_maestra

Amelia escribió dos tests de seguridad: uno que verifica que el proxy rechaza las cuatro operaciones con clave incorrecta lanzando `PermissionError`, y otro que verifica que tras expirar el timeout cualquier operación también lanza `PermissionError`.

Enrique implementó `ProxyGestorCredenciales` y el mecanismo de timeout con `_verificar_timeout()`, llamado al inicio de cada método del gestor.

Nikhil refactorizó separando la lógica de autenticación y simplificando las condiciones del timeout.

Ciclo 3 — listar_servicios, saludar

Enrique escribió dos tests: uno que añade varios servicios y comprueba que `listar_servicios()` los devuelve todos, y otro que verifica que `saludar()` retorna un mensaje no vacío que no contiene la clave maestra ni contraseñas.

Nikhil implementó ambas funciones, delegando el listado en el backend y devolviendo un mensaje estático en `saludar()`.

Gael refactorizó unificando el patrón de logging de todas las operaciones.

Ciclo 4 — password_no_almacenado_en_plano, exportar_credenciales_cifradas

Nikhil escribió dos tests de confidencialidad: uno que accede directamente al backend y verifica que el blob guardado no contiene la contraseña en claro, y otro que verifica que la exportación devuelve blobs cifrados y no contraseñas.

Gael implementó `exportar_credenciales_cifradas()`. Inicialmente se trabajó con cifrado autenticado, y en la versión final se integró `CifradorFernet` como estrategia principal de cifrado, devolviendo la copia del almacén con los blobs cifrados y registrando la operación en el log.

Sofía refactorizó añadiendo la llamada a `_verificar_timeout()` que faltaba en el método de exportación.

Ciclo 5 — deteccion_inyeccion_servicio, recuperar_credencial

Gael escribió dos tests: un fuzz con seis payloads maliciosos (SQL injection, XSS, path traversal, newline injection, comillas y cadena vacía) verificando que todos son rechazados; y otro que comprueba la recuperación normal y que un blob manipulado lanza `ValueError`.

Sofía implementó `validar_nombre_servicio()` con una expresión regular que solo acepta caracteres alfanuméricos, guion, punto y guion bajo, y añadió los decoradores `@require` de `icontract`.

Lara refactorizó unificando todos los validadores en `utils.py` y eliminando duplicaciones detectadas por SonarQube.

Ciclo 6 — fuzz_politica_passwords_debiles

Sofía escribió un fuzz test parametrizado con 20 contraseñas débiles usando `subTest`, cubriendo contraseñas demasiado cortas, sin mayúscula, sin minúscula, sin dígito, sin símbolo y contraseñas triviales. Cada caso es un subtest independiente.

Lara implementó `validar_password()` con la expresión regular que exige mínimo 12 caracteres, mayúscula, minúscula, dígito y símbolo especial.

Amelia refactorizó extrayendo cada criterio a una función auxiliar con nombre descriptivo para reducir la complejidad.

2.3. Práctica 6 — Diseño Seguro

En la Práctica 6 el trabajo se distribuyó en tres pares tester/implementador, uno por cada patrón:

Patrón	Tester	Implementador
Strategy (cifrado)	Lara	Amelia
Secure Proxy for API	Sofía	Enrique
Secure Strategy Factory	Nikhil	Gael

Lara escribió `test_cifrador.py` verificando que `CifradorFernet` cifra y descifra correctamente, que el blob no contiene la contraseña en claro, que una clave incorrecta o un blob manipulado lanzan `ValueError`, y que el gestor funciona con un cifrador alternativo sin modificar su código (OCP).

Amelia creó `cifrador.py` con la clase abstracta `CifradorBase` y la implementación `CifradorFernet`. Aplicó SRP (el módulo solo cifra), DIP (el gestor depende de la abstracción)

Sofía escribió `test_proxy.py` verificando que el proxy bloquea operaciones con clave incorrecta, las permite con clave correcta, y que cada intento queda registrado en el log (Registro de compromiso).

Enrique creó `proxy.py` con `ProxyGestorCredenciales`, que autentica cada petición antes de redirigirla al gestor real aplicando Mediación completa.

Nikhil escribió `test_factory.py` verificando que la fábrica devuelve `CifradorFernet` con credenciales válidas y que falla con rol o clave incorrectos (A prueba de fallos por defecto).

Gael creó `factory.py` con `CifradorFactory`, que selecciona el cifrador según el rol del cliente aplicando Mínimo privilegio.

3. Decisiones de diseño

3.1. Principios SOLID

SRP: cada módulo tiene una única responsabilidad. `cifrador.py` solo cifra, `proxy.py` solo autentica peticiones, `factory.py` solo selecciona cifradores.

OCP: para añadir un nuevo algoritmo de cifrado basta con crear la subclase en `cifrador.py` y registrarla en la fábrica, sin modificar la clase `GestorCredenciales`.

DIP: `GestorCredenciales` recibe sus dependencias como abstracciones en el constructor:

```
def __init__(self, clave_maestra, cifrado: CifradorBase =
    None, ...):
    self._cifrado = cifrado or CifradorFactory.
        obtener_cifrador("fernet")
```

3.2. Principios de seguridad

Economía de mecanismos: las validaciones están centralizadas en `utils.py` y cada módulo hace una sola cosa.

A prueba de fallos: cualquier operación con clave incorrecta, tipo desconocido o sesión expirada lanza una excepción. Nunca se permite acceso por omisión.

Mediación completa: toda operación realizada a través del proxy pasa por `_autenticar()`. Además, `_verificar_timeout()` se llama dentro del propio gestor como defensa adicional para aquellos escenarios donde se interactúa directamente con él (como en las pruebas unitarias).

Mínimo privilegio: la fábrica solo concede un cifrador a roles explícitamente autorizados. La exportación devuelve blobs cifrados, nunca contraseñas en claro.

Registro de compromiso: `LogSeguro` usa hash SHA-256 encadenado, haciendo detectable cualquier modificación retroactiva del log. El proxy registra todos los intentos, exitosos y fallidos.

3.3. Otras decisiones

Usamos `CifradorFernet` porque el guion lo menciona explícitamente y porque combina cifrado e integridad en un solo token. La derivación de clave usa PBKDF2-HMAC-SHA256 con 100 000 iteraciones para dificultar ataques de fuerza bruta. La versión final usa un salt aleatorio por credencial, almacenado junto al token cifrado, evitando reutilizar un salt fijo. Además, se mantiene compatibilidad con blobs antiguos.

Las precondiciones se declaran con `icontract` directamente en el código:

```
@icontract.require(lambda servicio: validar_nombre_servicio(
    servicio),
                    "Nombre de servicio invalido.")
@icontract.require(lambda password: validar_password(password
    ),
                    "Password no cumple la politica.")
def anyadir(self, servicio, usuario, password): ...
```

4. Conclusiones sobre S-TDD

El uso de S-TDD a lo largo de estas dos prácticas ha supuesto un cambio de mentalidad respecto al desarrollo que solíamos implementar. Trabajar con ciclos cortos y roles definidos nos ha enseñado que la seguridad no es algo que se añade al final, sino que debe estar presente desde la primera línea de código.

A continuación resumimos los aprendizajes más relevantes:

- **Seguridad por diseño:** integrar los requisitos de seguridad en la misma iteración funcional obligó al equipo a analizar las amenazas de forma proactiva y desde el inicio del ciclo de vida del desarrollo.
- **Pruebas sin sesgos:** la rotación de roles garantizó la objetividad. Al no tener la solución en mente, el tester diseñó escenarios más estrictos, logrando implementaciones más robustas.
- **Refactorización segura:** la madurez de la suite quedó demostrada al aislar módulos en ficheros independientes. La batería de pruebas actuó como una red de seguridad que validó la integridad estructural del software en tiempo real.

Concluyendo, S-TDD es una metodología altamente eficiente para proyectos donde la seguridad es un pilar crítico. El coste temporal de escribir los tests de forma anticipada se amortiza por completo al evitar el rediseño tardío del software.